

Python Variablen und Anweisungen

Variablen und Zuweisungen

Grundlagen der Variablen

- Eine **Variable** ist ein Name, der auf einen Wert verweist
- Erstellen einer Variablen mit **Zuweisungsanweisung**: `variable = wert`
- Beispiele:

```
n = 17                # Integer-Wert
pi = 3.141592653589793 # Fließkommazahl
message = 'And now for something completely different' # String
```

Verwendung von Variablen

- Variablenwerte können ausgegeben werden: `n` → 17
- Variablen in arithmetischen Ausdrücken: `n + 25` → 42
- Variablen in Funktionsaufrufen: `round(pi)` → 3
- Strings können mit `len()` gemessen werden: `len(message)` → 42

Regeln für Variablennamen

- Können beliebig lang sein
- Dürfen aus Buchstaben und Ziffern bestehen
- Dürfen nicht mit einer Ziffer beginnen
- Können Unterstriche enthalten (z.B. `your_name`, `air_speed_of_unladen_swallow`)
- Dürfen keine Sonderzeichen oder Leerzeichen enthalten
- Dürfen kein Schlüsselwort sein
- Sind normalerweise kleingeschrieben

Ungültige Variablennamen

- `76trombones` - beginnt mit einer Ziffer
- `million!` - enthält Sonderzeichen
- `class` - ist ein Schlüsselwort

Python-Schlüsselwörter (können nicht als Variablennamen verwendet werden)

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Zustandsdiagramme

- Visuelle Darstellung von Variablen und ihren Werten
- Beispiel:

```
n → 17
pi → 3.141592653589793
message → 'And now for something completely different'
```

Die import-Anweisung

Module importieren

- **Modul:** Eine Sammlung von Variablen und Funktionen
- `import math` lädt das math-Modul
- Nach dem Import kann auf Funktionen und Variablen des Moduls zugegriffen werden
- Syntax: `modulname.funktionsname()` oder `modulname.variablenname`
- Beispiel: `math.pi` gibt `3.141592653589793` zurück

Funktionen im math-Modul

- `math.sqrt(x)` - Quadratwurzel von x: `math.sqrt(25) → 5.0`
- `math.pow(x, y)` - x hoch y: `math.pow(5, 2) → 25.0`
- `math.sin(x)`, `math.cos(x)` - Trigonometrische Funktionen
- `math.exp(x)` - e hoch x (e^x)

Konstanten im math-Modul

- `math.pi` - Kreiszahl π (3.141592...)
- `math.e` - Eulersche Zahl e (2.718281...)

Punktoperator

- Der `.` (Punkt) zwischen Modulname und Funktion wird als **Punktoperator** bezeichnet
- Beispiel: `math.sqrt(25)` - `math` ist das Modul, `sqrt` ist die Funktion

Ausdrücke und Anweisungen

Ausdrücke

- Ein **Ausdruck** kann sein:
 - Ein einzelner Wert (Integer, Float, String)
 - Eine Kombination von Werten und Operatoren
 - Variablennamen oder Funktionsaufrufe
- Komplexe Ausdrücke: `19 + n + round(math.pi) * 2`
- **Auswertung:** Durchführung der Operationen in einem Ausdruck, um einen Wert zu berechnen

Anweisungen

- Eine **Anweisung** führt eine Aktion aus, hat aber keinen Wert
- Beispiele:
 - Zuweisungsanweisung: `n = 17`
 - Import-Anweisung: `import math`
- **Ausführung**: Die Befehle einer Anweisung befolgen

Die print-Funktion

Grundlegende Verwendung

- `print(wert)` gibt den Wert auf der Konsole aus
- `print('Text')` gibt Text aus
- Mehrere Ausdrücke ausgeben:

```
print(n+2)    # Ausgabe: 19
print(n+3)    # Ausgabe: 20
```

Mehrere Werte in einer Zeile ausgeben

- `print(ausdruck1, ausdruck2, ...)` gibt mehrere Werte aus, getrennt durch Leerzeichen
- Beispiel: `print('Value of pi is', math.pi)` → Value of pi is 3.141592653589793

Funktionsargumente

Was sind Argumente?

- **Argument**: Ein Wert, der einer Funktion bei ihrem Aufruf übergeben wird
- Argumente stehen in Klammern nach dem Funktionsnamen

Arten von Funktionsargumenten

- Funktionen mit einem Argument: `int('101') → 101`
- Funktionen mit zwei Argumenten: `math.pow(5, 2) → 25.0`
- Funktionen mit optionalen Argumenten:
 - `int('101', 2) → 5` (zweites Argument gibt die Basis an)
 - `round(math.pi, 3) → 3.142` (zweites Argument gibt die Anzahl der Nachkommastellen an)
- Funktionen mit variabler Anzahl von Argumenten: `print('Any', 'number', 'of', 'arguments')`

Fehler bei Argumenten

- Zu viele Argumente: `float('123.0', 2) → TypeError`
- Zu wenige Argumente: `math.pow(2) → TypeError`
- Falscher Argumenttyp: `math.sqrt('123') → TypeError`

Kommentare

Syntax und Verwendung

- Beginnen mit dem Doppelkreuz #
- Alles ab dem # bis zum Zeilenende wird vom Interpreter ignoriert
- Einfügen von Kommentaren:

```
# Dies ist ein Kommentar auf einer eigenen Zeile  
x = 5 # Dies ist ein Kommentar am Ende einer Codezeile
```

Gute Kommentare schreiben

- Erklären das "Warum", nicht das "Was"
- Schlechter Kommentar: `v = 8 # assign 8 to v` (unnötig, da offensichtlich)
- Guter Kommentar: `v = 8 # velocity in miles per hour` (gibt zusätzliche Information)
- Gute Variablennamen können Kommentare überflüssig machen

Fehlerarten und Debugging

Syntaxfehler

- Fehler in der Struktur des Programms
- Programm wird nicht ausgeführt
- Beispiele:
 - Ungültige Variablennamen: `million! = 1000000`
 - Verwendung von Schlüsselwörtern als Variablennamen: `class = 'Python'`

Laufzeitfehler (Ausnahmen)

- Treten während der Ausführung des Programms auf
- Führen zum Abbruch des Programms
- Beispiel: `'126' / 3` → `TypeError`, da ein String nicht durch eine Zahl geteilt werden kann

Semantische Fehler

- Programm läuft ohne Fehlermeldung, tut aber nicht das Gewünschte
- Beispiel: `1 + 3 / 2` ergibt `2.5` statt `2.0` (bei Berechnung des Durchschnitts von 1 und 3)
- Schwierig zu finden, erfordern logisches Denken

Übungsbeispiele

Beispiel 1: Kugelvolumen berechnen

```
import math  
  
# Radius in Zentimetern  
radius = 5  
  
# Volumen in Kubikzentimetern berechnen mit  $V = \frac{4}{3} * \pi * r^3$ 
```

```
volume = (4/3) * math.pi * radius**3

print("Das Volumen einer Kugel mit Radius", radius, "cm beträgt", volume, "cm³")
```

Beispiel 2: Trigonometrische Identität überprüfen

```
import math

x = 42 # Beliebiger Wert für x

# Berechnen der Summe von (cos x)² + (sin x)² (sollte 1 ergeben)
result = math.cos(x)**2 + math.sin(x)**2

print("(cos x)² + (sin x)² =", result) # Sollte ungefähr 1 sein
```

Beispiel 3: e^2 auf verschiedene Arten berechnen

```
import math

# Methode 1: Mit Potenzierungsoperator
result1 = math.e ** 2

# Methode 2: Mit math.pow
result2 = math.pow(math.e, 2)

# Methode 3: Mit math.exp
result3 = math.exp(2)

print("e² mit ** Operator:", result1)
print("e² mit math.pow():", result2)
print("e² mit math.exp():", result3)
```

Glossar

Begriff	Beschreibung
Variable	Ein Name, der auf einen Wert verweist
Zuweisungsanweisung	Eine Anweisung, die einer Variablen einen Wert zuweist
Zustandsdiagramm	Eine grafische Darstellung von Variablen und ihrer Werte
Schlüsselwort	Ein spezielles Wort für die Definition der Programmstruktur
import-Anweisung	Eine Anweisung, die eine Moduldatei lädt
Modul	Eine Sammlung von Variablen und Funktionen
Punktoperator	Der Operator (.) für den Zugriff auf Funktionen in einem Modul

Begriff	Beschreibung
Anweisung	Eine oder mehr Codezeilen, die für einen Befehl stehen
Auswertung	Durchführung der Operationen in einem Ausdruck, um einen Wert zu berechnen
Ausführung	Die Befehle einer Anweisung befolgen
Argument	Ein Wert, der einer Funktion bei ihrem Aufruf übergeben wird
Kommentar	Text im Programm, der vom Interpreter ignoriert wird
Laufzeitfehler	Ein Fehler, der während der Ausführung auftritt
Ausnahme	Ein Fehler, der während der Ausführung des Programms auftritt
Semantischer Fehler	Ein Fehler, der dafür sorgt, dass das Programm nicht das tut, was es soll